

TP 2 - Modelado Estadístico

Allami Florencia - Vanotti Franco

Exploración inicial

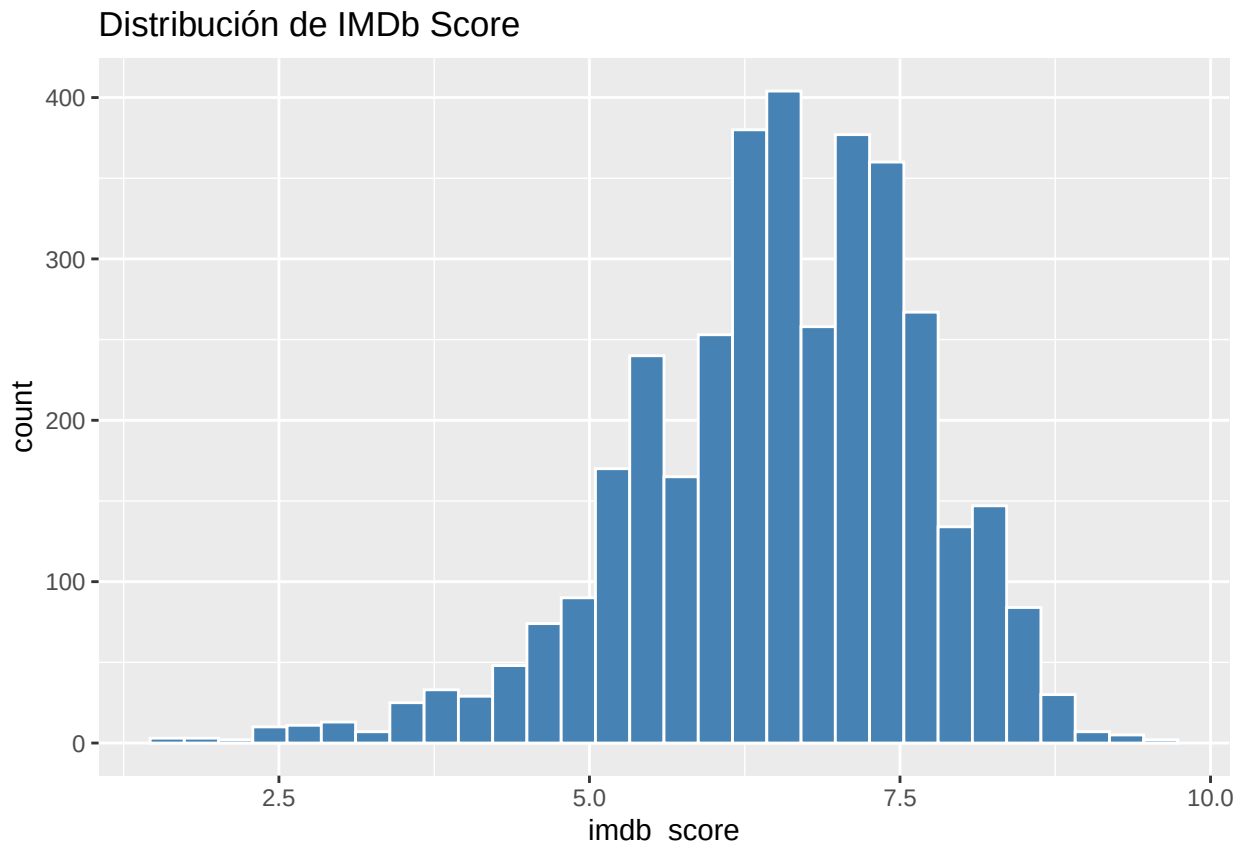
Como primer paso se realiza una inspección general del conjunto de datos para conocer su estructura, el tipo de las variables disponibles y la presencia de valores faltantes. Esta información resulta útil para decidir qué variables pueden utilizarse posteriormente en el análisis y qué preprocesamiento será necesario realizar.

El conjunto de datos contiene **3631 títulos** y **12 variables**, entre las que se incluyen información descriptiva (título, descripción, géneros y país de producción), características del contenido (duración, año de lanzamiento y clasificación por edad) y la variable objetivo **IMDb Score**.

No se observan valores faltantes en la mayoría de las variables. Sin embargo, la variable **seasons** presenta una gran cantidad de valores ausentes, lo cual es esperable ya que únicamente aplica a series y no a películas. La variable **imdb_votes** posee únicamente 13 valores faltantes.

Distribución del puntaje de IMDb

Antes de estudiar la relación entre las covariables y el puntaje de IMDb, resulta útil analizar la distribución de la variable respuesta. Para ello se construye un histograma del puntaje de IMDb.



La distribución presenta una forma aproximadamente unimodal, concentrándose la mayor parte de los títulos entre puntajes **6 y 7.5**. Existen pocos títulos con puntajes extremadamente bajos o muy cercanos al máximo, por lo que la distribución presenta una ligera asimetría hacia la izquierda.

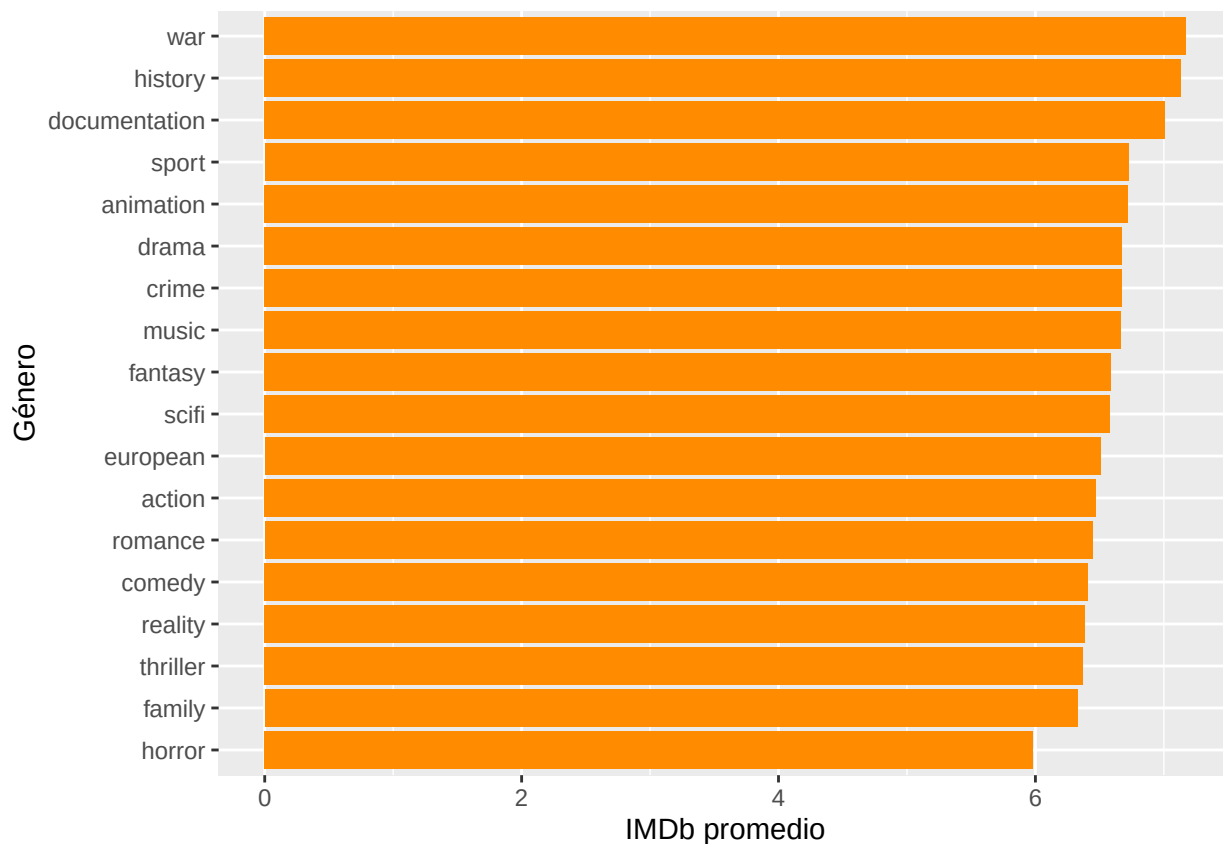
Esto indica que la mayoría de los contenidos poseen una valoración intermedia-alta, mientras que los títulos muy malos o excelentes representan una proporción reducida del conjunto de datos.

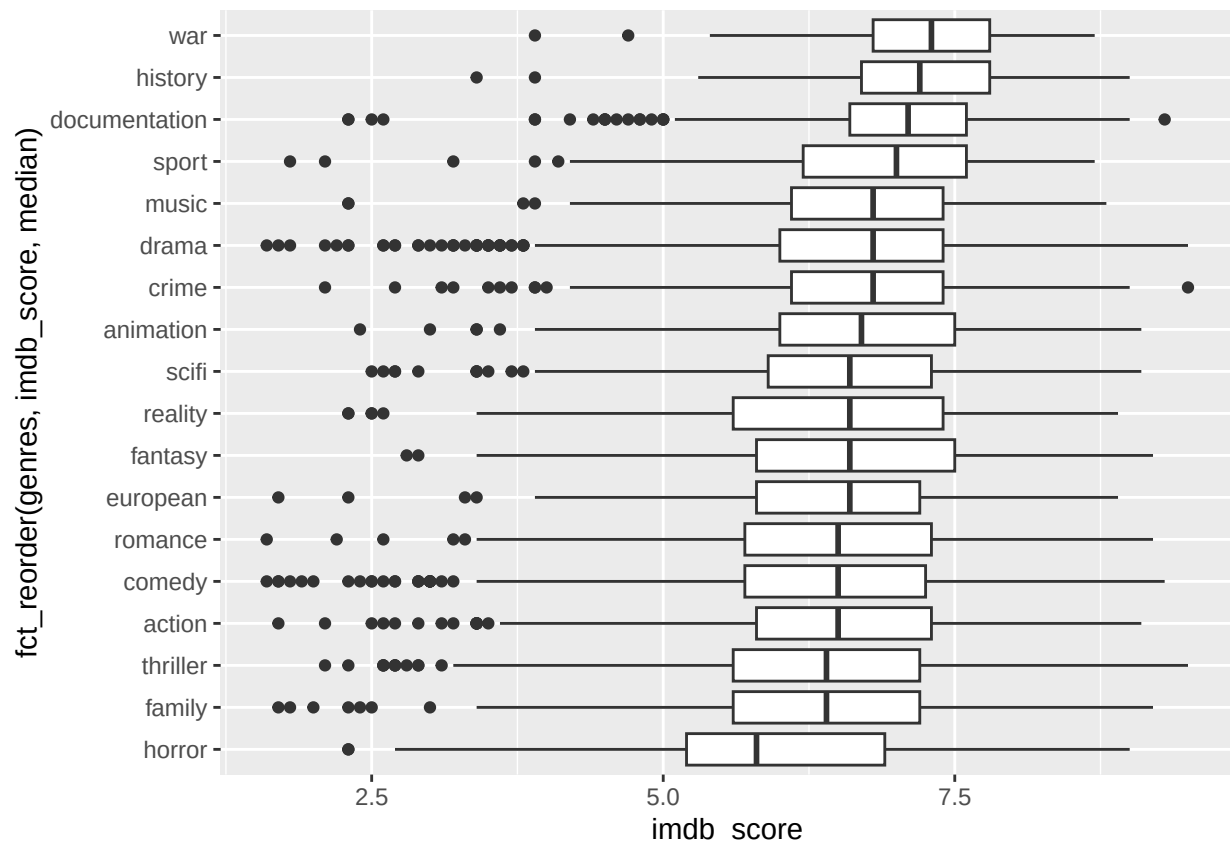
Relación entre el género y el puntaje de IMDb

La variable **genres** se encuentra almacenada como una lista codificada en formato texto, por lo que primero es necesario transformarla para obtener un registro por cada género asociado a un título. De esta forma será posible calcular estadísticas descriptivas para cada género individualmente.

Una vez separado cada género en una observación independiente, los datos quedan preparados para calcular el puntaje promedio y estudiar la distribución del puntaje dentro de cada categoría.

Para responder el inciso (a), se calcula el puntaje promedio de IMDb para cada género y posteriormente se analizan las distribuciones mediante boxplots. Con el objetivo de evitar conclusiones basadas en muy pocas observaciones, únicamente se consideran géneros presentes en al menos 50 títulos.





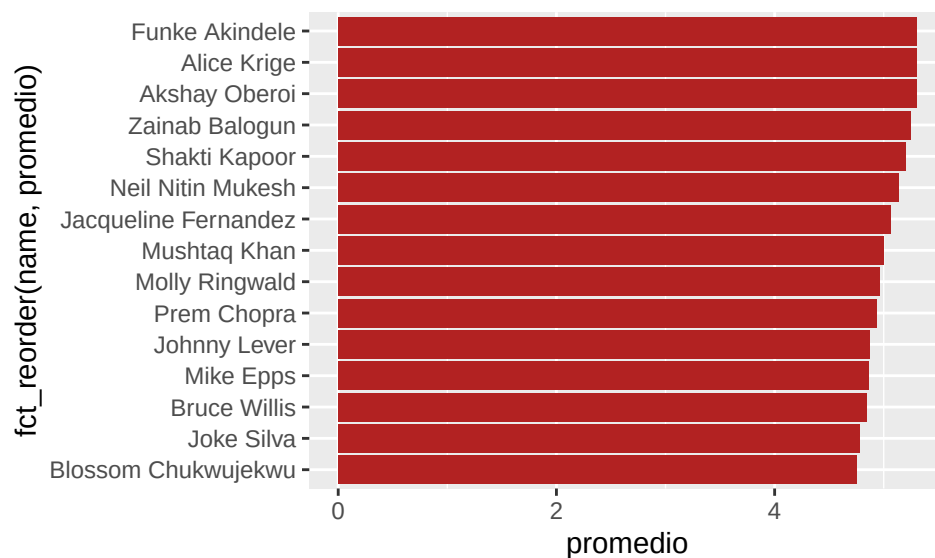
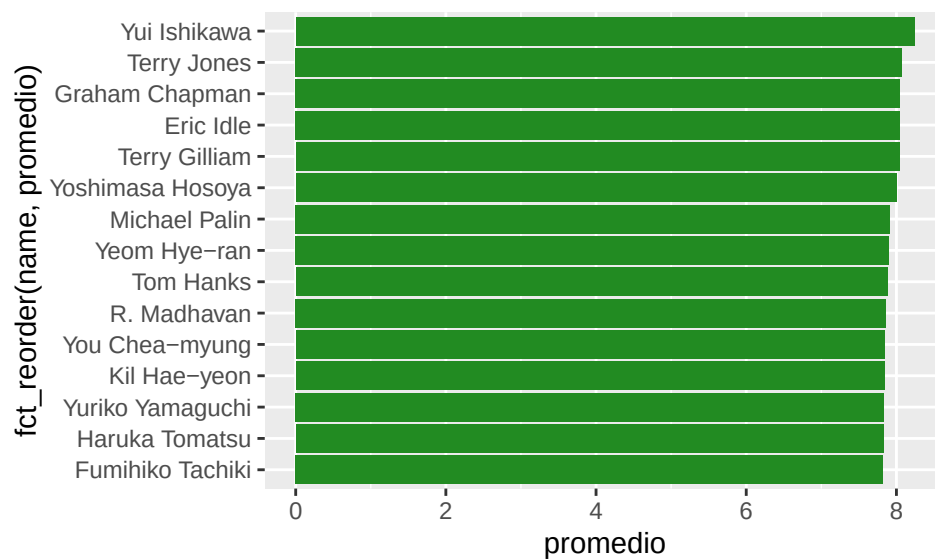
Se observa que los géneros **War**, **History** y **Documentation** presentan, en promedio, los puntajes más elevados, mientras que **Horror** y **Family** aparecen entre los de menor promedio.

Sin embargo, los diagramas de caja muestran un importante solapamiento entre las distribuciones de los distintos géneros, lo que indica que el género por sí solo no determina completamente la calificación de un título. Aun así, existen diferencias sistemáticas que sugieren que esta variable podría aportar información útil para la predicción del puntaje.

Actores asociados con mejores y peores puntajes

Para estudiar la influencia del elenco, se unen los datos de actores con los puntajes de IMDb mediante el identificador del título. Luego se calcula el puntaje promedio obtenido por cada actor.

Con el fin de reducir el efecto del azar, solamente se consideran actores que hayan participado en al menos cinco títulos del conjunto de entrenamiento.

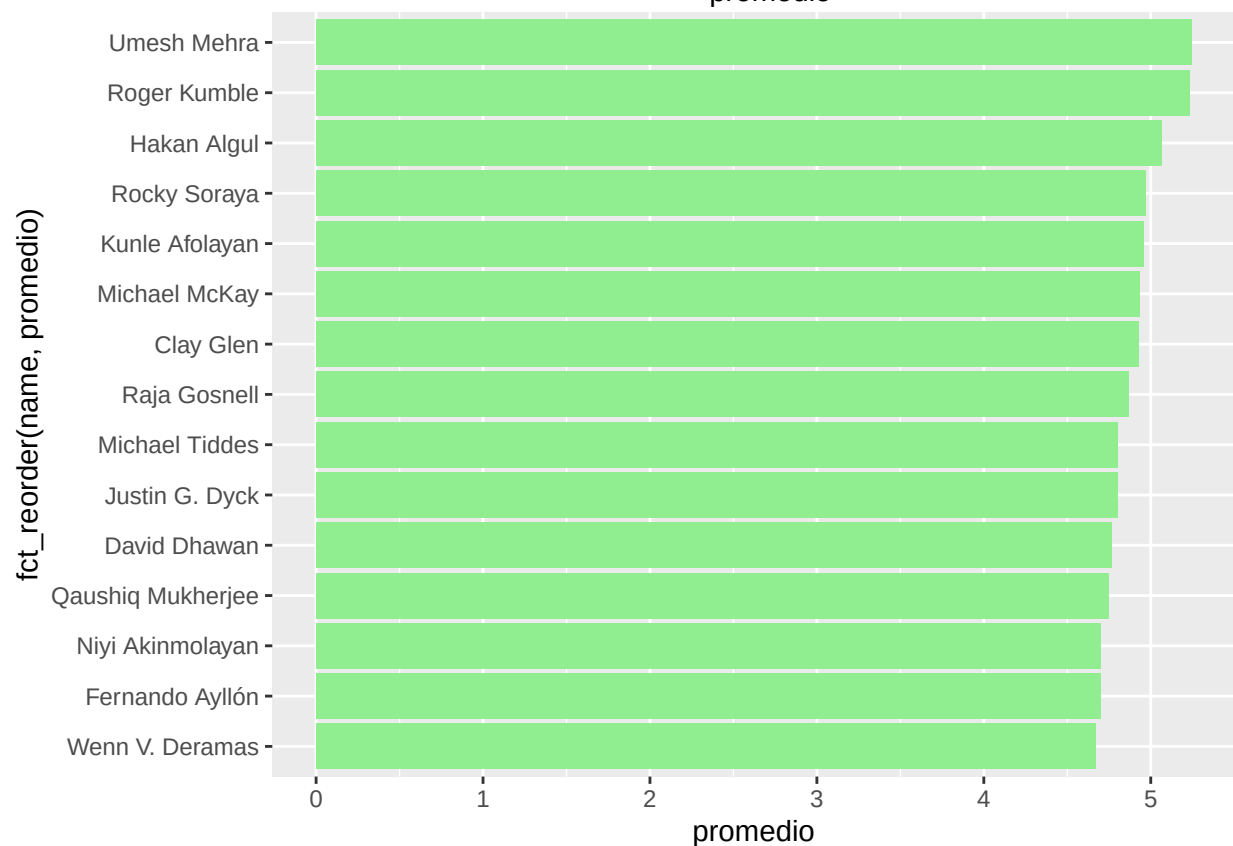
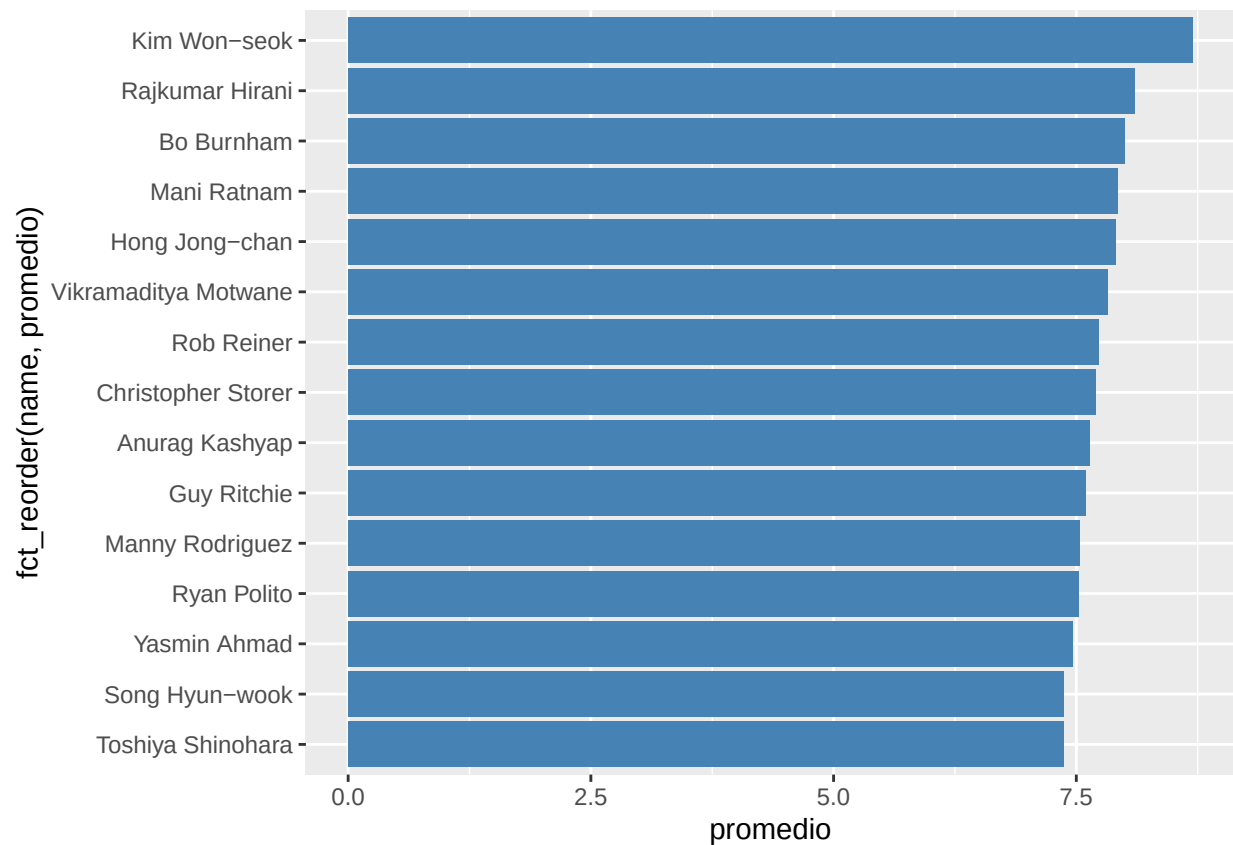


Los gráficos muestran los actores con mayor y menor puntaje promedio. Debe destacarse que estos resultados representan una asociación y no una relación causal. Un actor puede aparecer con un promedio elevado simplemente porque ha participado en un conjunto reducido de producciones muy bien valoradas.

No obstante, este análisis permite identificar que la participación de determinados actores podría aportar información relevante para estimar el puntaje de IMDb.

Directores asociados con mejores y peores puntajes

Se realiza un procedimiento análogo para los directores, calculando el puntaje promedio de las producciones dirigidas por cada uno de ellos. En este caso se consideran únicamente aquellos directores con al menos tres títulos en el conjunto de datos.



Se observan diferencias importantes entre los promedios de los distintos directores. Algunos presentan

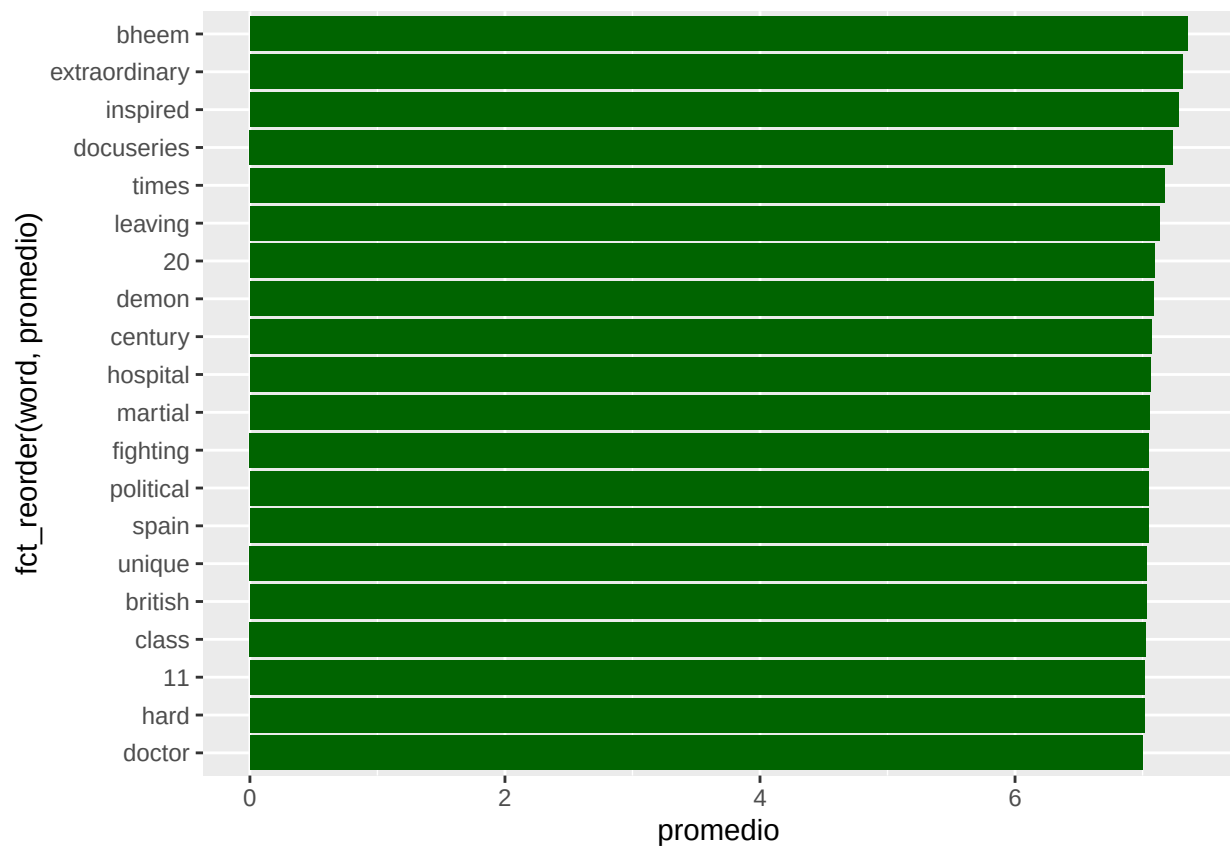
promedios superiores a 8 puntos, mientras que otros se encuentran por debajo de 5.

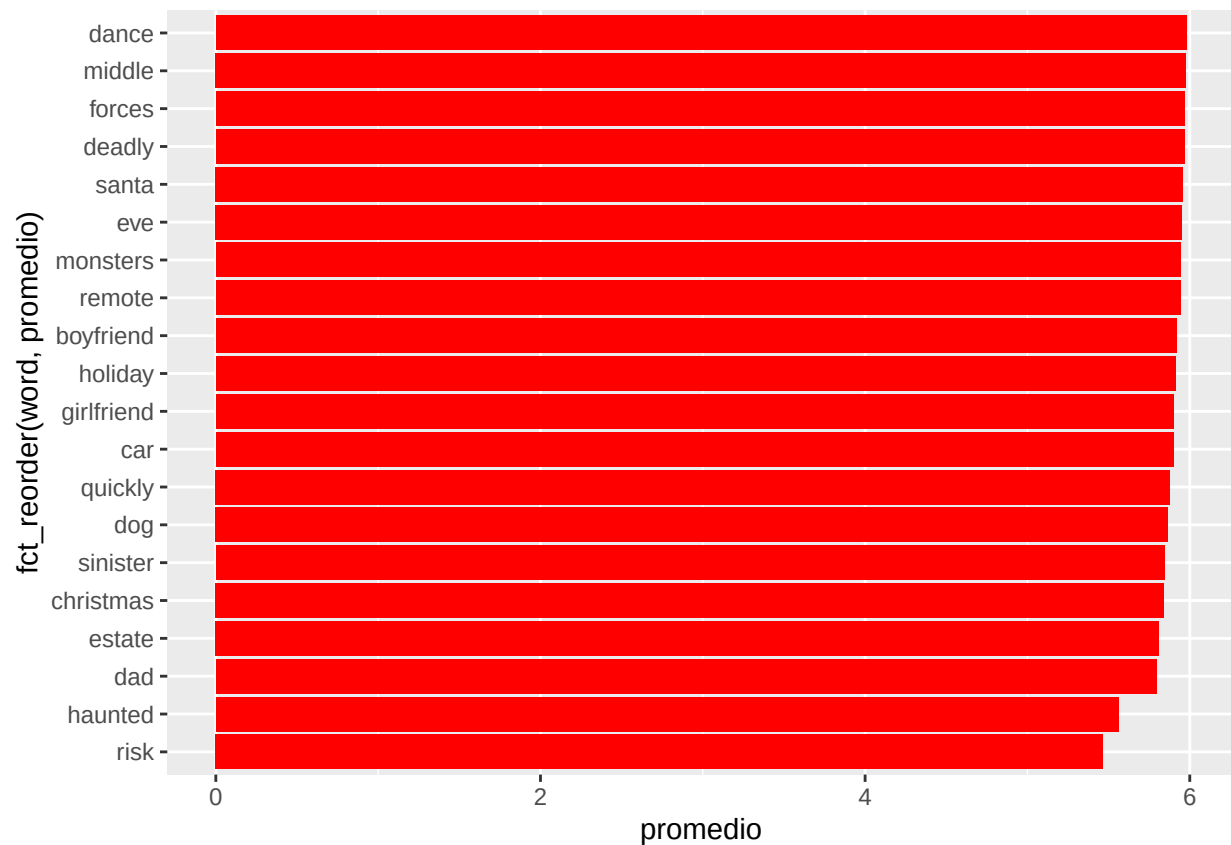
Al igual que en el caso de los actores, estos resultados deben interpretarse con cautela, ya que el promedio depende tanto de la calidad de las producciones como de la cantidad de títulos considerados. Sin embargo, la identidad del director parece contener información útil para explicar el puntaje obtenido por una producción.

Palabras de la descripción asociadas al puntaje

Para responder el inciso (c), se analiza el texto de la descripción de cada título utilizando técnicas básicas de procesamiento de lenguaje natural. En primer lugar, las descripciones se dividen en palabras individuales (*tokenización*) y posteriormente se eliminan las palabras vacías (*stop words*), ya que suelen aportar poca información semántica.

Finalmente, se calcula el puntaje promedio asociado a cada palabra, considerando únicamente aquellas que aparecen al menos 20 veces en el conjunto de datos.





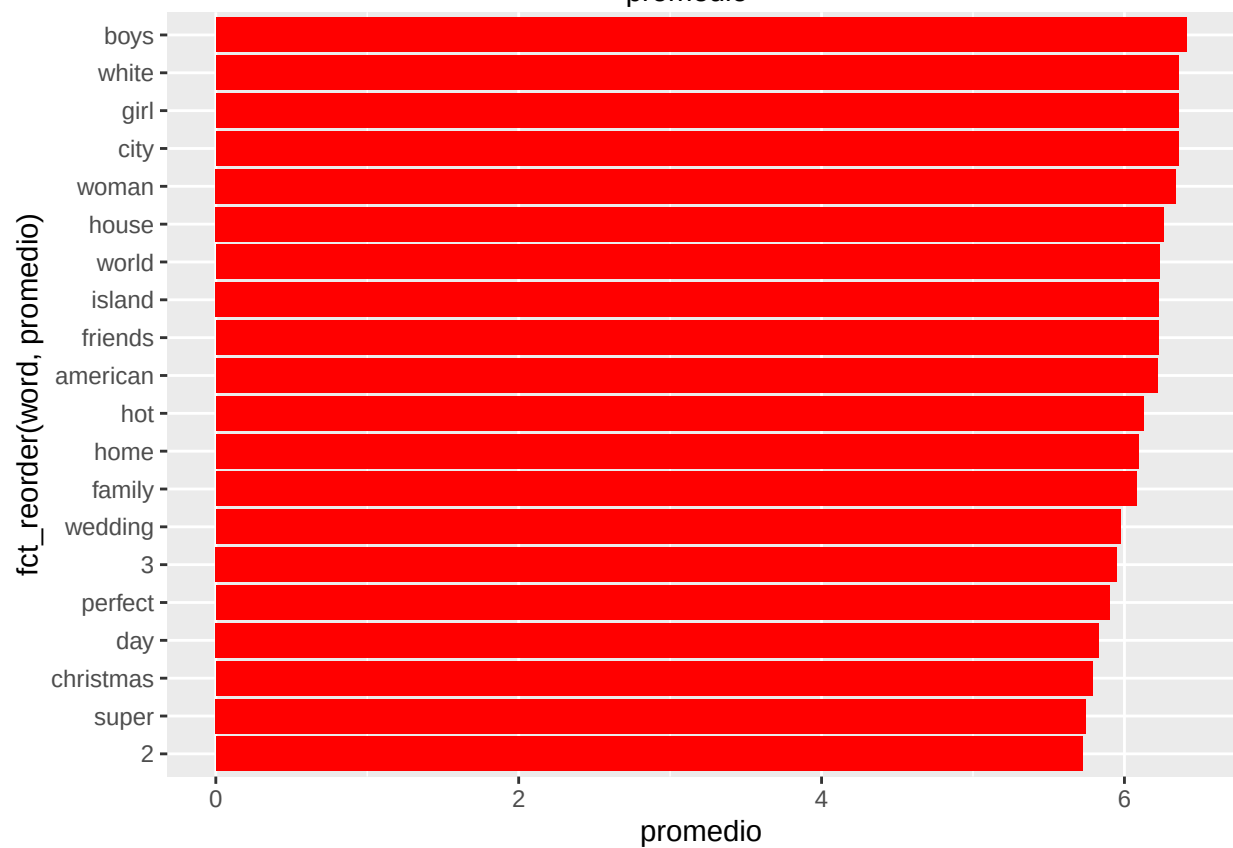
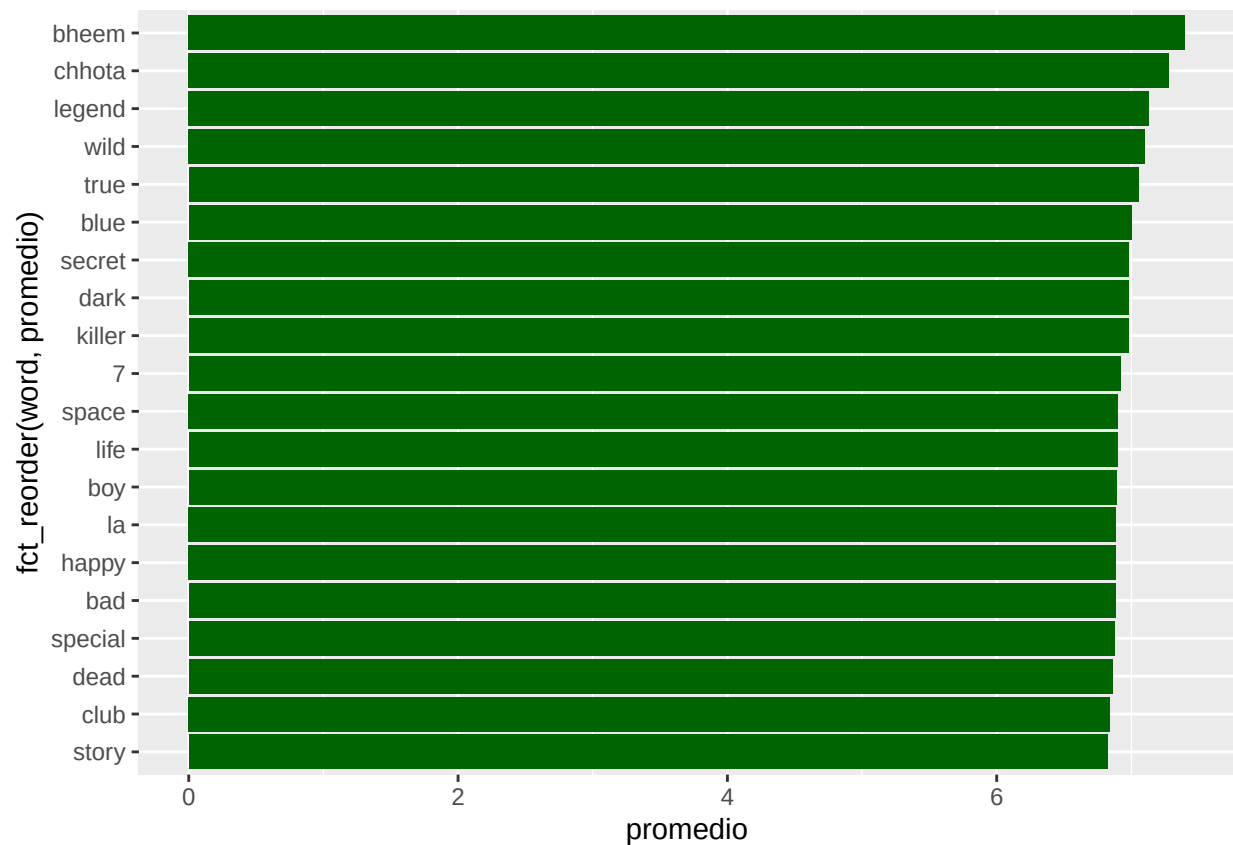
Se identifican palabras cuya presencia se asocia con puntajes promedio más altos y otras con puntajes inferiores. Sin embargo, estas asociaciones no deben interpretarse como relaciones causales, sino como indicadores de tendencias presentes en el conjunto de datos.

Este análisis sugiere que el contenido textual de las descripciones contiene información predictiva que podría ser aprovechada posteriormente mediante técnicas de representación de texto.

Palabras del título asociadas al puntaje

De manera similar al análisis anterior, se estudian las palabras presentes en el nombre de cada título. Luego de tokenizar los títulos y eliminar las palabras vacías, se calcula el puntaje promedio asociado a cada palabra, considerando únicamente aquellas que aparecen al menos diez veces.

```
## Joining with `by = join_by(word)`
```



El análisis muestra que ciertas palabras aparecen con mayor frecuencia en títulos con puntajes elevados,

mientras que otras se asocian con producciones de menor valoración promedio.

Si bien estas diferencias pueden deberse parcialmente al tipo de contenido o al género de las producciones, los resultados indican que el título también contiene información potencialmente útil para construir modelos predictivos.

2. Causalidad.

El efecto de **Comedia** sobre **Score** está mediado por **Duración**, que crea un camino como cadena. Además, está afectado por **País**, que crea un camino como confounder. Al condicionar por **Duración**, que es collider entre **Año** y **Comedia**, se abre el camino **Comedia** -> **Duración** <- **Año** -> **Score**, por lo que hay que también condicionar por **Año**. Entonces, la forma más clara de medir el efecto causal directo de **Comedia** sobre **Score** es condicionando por **Duración**, **País** y **Año**. Dado que todas estas variables son necesarias, y no hay otras en el DAG, este es el único conjunto por el que podemos condicionar para medir el efecto.

Luego, vamos a medir la asociación entre las variables, realizando una regresión lineal. Primero hacemos una regresión en la que ponemos a **Score** en función de las variables **Comedia**, **País**, **Año** y **Duración**. Al condicionar por este conjunto (además de **Comedia**), logro independencia entre el **Score** si fuera **Comedia** y la variable **Comedia**. Esto nos permite decir que el índice de asociación es igual al efecto causal promedio.

Por otro lado, al hacer una regresión entre **Comedia** y **Score**, podemos calcular la asociación entre las variables.

El coeficiente asociado a **Comedia** en el modelo más simple nos dio -0.20632 , mientras que en el modelo con más variables dio -0.2352186 . Esto implica que el que una película sea de comedia generalmente causa que tenga peor **Score**. Otras variables afectan esta probabilidad, pero no demasiado.

3. Selección de modelos.

Preprocesamiento Vamos a hacer 5 modelos: regresión lineal, efectos mixtos, regresión con splines, Random Forest y XGBoost. Vamos a medir su efectividad con cross validation de 5 capas.

Consideramos que tenemos variables numéricas (`runtime`, `release_year`, `seasons`, `imdb_votes`), categóricas (`type`, `age_certification`, `genres`, `production_countries`) y de texto (`title`, `description`). Además, tenemos el dataset de créditos de las películas. Para poder aprovechar esta información en nuestros modelos, vamos a tener que procesar los datos.

Primero que nada, decidimos preparar los datos para ser usados. En el dataset original tenemos las columnas `production_countries` y `genres`, que contienen arrays con los valores que aplican a la película. Esto es difícil de trabajar, y se pierde la noción de que una película hecha en ['GB'] está relacionada con una hecha en ['GB', 'FR']. Luego, cambiamos el dataset para que tenga una columna por categoría, y la variable es binaria según si aplica o no. Para las películas en las que no se sabía, se marcó la columna **Unknown**, y si la película pertenecía a un género o país poco popular (con menos de 30 apariciones en el dataset), se lo consideró como 'Otro'.

Por otro lado, queríamos incorporar algo de los datos de la tabla de créditos. Al fin y al cabo, los actores y el director presentes en una película influyen en su éxito. Primero agregamos a nuestras métricas por película la cantidad de actores y directores que la hicieron. Por otro lado, estudiamos la totalidad de nuestros datos (nuestro universo), y definimos al grupo de actores y directores “populares” como el 10% que participó en más películas. Así generamos otra métrica, cantidad de actores (o directores) populares por película.

También queríamos tener alguna medida de actores (y directores) “buenos”, y usar esa información en nuestras predicciones. En este caso, decidimos calcular para cada persona su **Score** promedio, sacado de las películas en las que participó, y luego para cada película el promedio de los **Scores** de sus actores y directores, y el **Score** máximo. Como esta información depende de nuestra variable a calcular, no podemos usar el total de la información de nuestro dataset y después usar cross validation, porque estaríamos usando la respuesta para predecir la respuesta. Por eso, estas variables las vamos a calcular para cada fold.

```
#####  
# 5. Placeholders para las features que SÍ usan imdb_score
```

```

# (se llenan de verdad más abajo, fold por fold; acá solo
# necesitamos que existan como columnas para que recipe()
# las registre como predictoras)
#=====

titles <- titles %>%
  mutate(
    actor_mean_score = 0,
    actor_max_score = 0,
    director_mean_score = 0,
    director_max_score = 0
  )

```

Por último, para trabajar con los campos de texto de nuestro dataset, en este caso `title` y `descripción`, decidimos usar TF-IDF. El proceso es así: primero se separa el campo en palabras (tokeniza). Luego se extraen las palabras más comunes (como “in” o “the”), y se las lleva a su raíz (por ejemplo, los plurales se vuelven singulares). Luego nos quedamos con las palabras más comunes, y les aplicamos el algoritmo TF-IDF. Este algoritmo calcula la frecuencia de una palabra en su propio texto, y la compara con su frecuencia en el total de los textos. De esta forma determina las palabras clave de un texto, y usa esto para sus predicciones.

Como las variables basadas en el `imdb_score` utilizan la variable respuesta para construirse, es necesario recalcularlas dentro de cada fold de la validación cruzada para evitar fuga de información. Además, incluso dentro del conjunto de entrenamiento de un fold, el promedio asociado a un actor o director no debe incluir la película que se está intentando predecir. Para resolver esto utilizamos una estrategia leave-one-out: al calcular el promedio de una película de entrenamiento, primero se excluye su propio `imdb_score` del promedio de sus actores y directores.

Una vez definidas las funciones para construir las variables derivadas de actores y directores, preparamos los folds de la validación cruzada. En cada fold se calculan estas variables utilizando únicamente la información disponible en el conjunto de entrenamiento, y luego se aplica la misma receta de preprocesamiento tanto al conjunto de entrenamiento como al de evaluación. Como este procedimiento es independiente del modelo que se vaya a ajustar, los folds se preparan una única vez y luego se reutilizan en todos los modelos.

Con los folds ya preparados implementamos una función general para evaluar modelos mediante validación cruzada. Esta función entrena el modelo en cada fold, obtiene las predicciones sobre el conjunto de evaluación y calcula las métricas de interés (RMSE, MSE y R^2). De esta forma todos los modelos son evaluados exactamente sobre las mismas particiones de los datos, haciendo que la comparación entre ellos sea justa.

Los modelos que poseen hiperparámetros (como Random Forest y XGBoost) requieren una etapa adicional para seleccionar su configuración. Para ello implementamos una búsqueda en grilla (grid search), donde se evalúan distintas combinaciones de hiperparámetros utilizando un conjunto de folds independiente del empleado para reportar el desempeño final. Una vez seleccionada la mejor combinación, el modelo correspondiente se vuelve a evaluar sobre los folds principales para compararlo con el resto de los modelos.

Regresión lineal Como línea de base, ajustamos una regresión lineal múltiple utilizando todas las variables disponibles luego del preprocesamiento. Este modelo supone que el efecto de cada predictor sobre el `imdb_score` es lineal y aditivo, por lo que sirve como referencia para evaluar si modelos más complejos logran mejorar el desempeño predictivo.

La validación cruzada arrojó un MSE promedio de 1.162, equivalente a un RMSE de 1.076, con un R^2 de 0.221. Esto indica que el modelo logra explicar aproximadamente un 22% de la variabilidad del puntaje de IMDB y comete, en promedio, un error cercano a 1.08 puntos sobre la escala de puntuación.

GAM A continuación ajustamos un Modelo Aditivo Generalizado (GAM). A diferencia de la regresión lineal, este modelo permite que algunas variables numéricas tengan una relación no lineal con el `imdb_score` mediante funciones suaves (splines), mientras que el resto de las variables (como las dummies y las variables

obtenidas mediante TF-IDF) se incorporan de forma lineal. Esto permite capturar patrones más complejos sin perder interpretabilidad.

La validación cruzada obtuvo un MSE promedio de 1.105, un RMSE de 1.049 y un R^2 de 0.253. Comparado con la regresión lineal, el GAM redujo el error de predicción y aumentó la proporción de variabilidad explicada, lo que sugiere que existen relaciones no lineales entre algunas variables numéricas y el puntaje de IMDB que la regresión lineal no logra capturar.

Modelo mixto También evaluamos un modelo de efectos mixtos. En este caso se utilizaron todas las variables como efectos fijos y se agregó un intercepto aleatorio asociado al director principal de cada película. La idea es capturar posibles diferencias sistemáticas entre directores que no puedan explicarse únicamente mediante las demás variables incluidas en el modelo.

La validación cruzada produjo un MSE promedio de 1.162, un RMSE de 1.076 y un R^2 de 0.221. Estos resultados son prácticamente idénticos a los obtenidos con la regresión lineal, lo que indica que incorporar un intercepto aleatorio para el director principal no aporta una mejora apreciable en la capacidad predictiva del modelo.

Una posible explicación es que la mayoría de los directores aparecen en muy pocas películas dentro del conjunto de datos, por lo que el modelo dispone de poca información para estimar con precisión un efecto aleatorio específico para cada uno.

Random Forest El modelo Random Forest es un modelo que aprovecha la robustez del árbol de decisión, y le agrega varianza. Funciona de la siguiente forma. Primero se toma la muestra original. Digamos que vamos a hacer t árboles. Entonces, hacemos t versiones Bootstrap de la muestra, es decir, si la muestra tiene n observaciones, tomamos n observaciones con reposición. Luego, para cada muestra bootstrapeada construimos un árbol. Para contruirlo, en cada paso vamos a tomar un subconjunto de tamaño m de las p variables que componen la observación, y vamos a generar el nodo que mejor divide la muestra en función de esas variables, tomando distintas variables en cada paso del árbol hasta completarlo. Finalmente tenemos t árboles distintos.

Para predecir un nuevo valor, cada árbol hace su predicción, y luego se promedia el resultado (o, si fuera categórica, se elige la categoría con más votos).

Para nuestro modelo, decidimos hacer 500 árboles, pero la cantidad $mtry$ de variables por paso, min_n , la mínima cantidad de residuos necesarios en un nodo para poder generarle hojas, $max.depth$ y $sample.fraction$ se seleccionaron automáticamente mediante una búsqueda sobre una grilla de valores, utilizando validación cruzada sobre un conjunto de datos separado del utilizado para reportar el desempeño final. De esta forma evitamos que la selección de hiperparámetros influya en la estimación del error.

Si m es demasiado bajo, los árboles van a ser poco precisos, y no van a servir, pero si es demasiado grande, van a ser demasiado similares, y se pierde la ventaja del random forest. Por otro lado, si min_n es demasiado bajo, corremos el riesgo de sobreajustar, mientras que si es muy alto, desajustamos. Los mejores valores encontrados fueron $mtry = 78$, $min_n = 8$, $max.depth = 26$ y $sample.fraction = 0.975$. El valor elevado de $mtry$ puede parecer llamativo, ya que habitualmente en problemas de regresión se utilizan aproximadamente un tercio de las variables. Sin embargo, en este caso gran parte de los predictores provienen de variables dummy y de la representación TF-IDF del texto, por lo que muchas de ellas contienen información limitada de manera individual. Permitir que cada división del árbol considere un mayor número de variables aumenta las probabilidades de encontrar particiones informativas.

Una vez seleccionados los hiperparámetros, el modelo se volvió a evaluar utilizando los mismos folds empleados para comparar todos los modelos anteriores. El Random Forest obtuvo un MSE promedio de 0.850, un RMSE de 0.920 y un R^2 de 0.393, mejorando de manera considerable a la regresión lineal, al modelo mixto y al GAM. Esto sugiere que existen relaciones complejas e interacciones entre las variables que los modelos lineales no logran capturar, mientras que el Random Forest puede modelarlas de manera flexible.

XGBoost XGBoost es otro modelo basado en árboles de decisión, pero su estrategia es distinta a la del Random Forest. Mientras que en un Random Forest todos los árboles se construyen de manera independiente y luego se promedian sus predicciones, en XGBoost los árboles se generan de forma secuencial. El primer árbol parte de una predicción inicial, que en este caso es el promedio de los Scores, y cada árbol posterior intenta corregir los errores cometidos por los anteriores. De esta manera, el modelo va refinando progresivamente sus predicciones.

Nuestros árboles no van a ser completos. Para decidir si un nodo amerita dividirse en dos hojas, vamos a calcular los índices de similitud de los nodos, y calcular la ganancia de similitud que nos dan las hojas. Si la ganancia es suficientemente buena (límite definido por el modelador), mantenemos las hojas. Hacemos las divisiones maximizando la similitud.

La similitud de un nodo la medimos como la suma de los residuos en ese nodo (si hay residuos positivos y negativos se cancelan), elevado al cuadrado y dividido por la cantidad de residuos más λ . La ganancia es la suma de la similitud de las hojas menos la similitud de la raíz. λ es el parámetro de regularización. Si λ es 0, los nodos con pocos residuos van a tener similitudes muy altas respecto al resto. Aumentar λ disminuye esta ventaja.

Otra consideración importante es que la construcción de árboles es iterativa. La primera predicción es el promedio de los scores. El primer árbol contiene en sus nodos los residuos respecto a esa predicción. El segundo árbol contiene los residuos respecto a la predicción conjunta del promedio y el primer árbol. Así, cada árbol corrige sobre los anteriores. Para hacer esta aproximación al resultado correcto cada vez más gradual, se utiliza una tasa de aprendizaje. Así, una predicción es el promedio más la suma de los residuos de los árboles multiplicados por la tasa de aprendizaje, un número entre 0 y 1.

Para evitar el sobreajuste, XGBoost incorpora distintos mecanismos de regularización. Entre ellos se encuentran la profundidad máxima de los árboles (`tree_depth`), la tasa de aprendizaje (`learn_rate`), que controla cuánto contribuye cada árbol nuevo a la predicción final, el parámetro `loss_reduction`, que exige una mejora mínima antes de realizar una nueva división en un árbol, y `sample_size`, que determina qué proporción de las observaciones se utiliza para entrenar cada árbol. Además, al igual que en Random Forest, el parámetro `mtry` controla cuántas variables candidatas se consideran en cada división.

Los hiperparámetros se seleccionaron mediante validación cruzada utilizando el conjunto de datos reservado para tuning, de forma independiente al conjunto empleado para evaluar el desempeño final del modelo. Los mejores valores obtenidos fueron `mtry` = 104, `trees` = 996, `tree_depth` = 6, `learn_rate` = 0.026, `loss_reduction` = 2.67 y `sample_size` = 0.904. Estos valores indican que el modelo prefiere construir una gran cantidad de árboles relativamente poco profundos, realizando pequeñas correcciones en cada iteración mediante una tasa de aprendizaje baja, una estrategia característica de los métodos de gradient boosting.

Finalmente, el modelo se evaluó utilizando los mismos folds empleados para el resto de los modelos, obteniendo un MSE promedio de 0.836, un RMSE de 0.913 y un R^2 de 0.396. Estos resultados fueron los mejores de todos los modelos evaluados, aunque la mejora respecto del Random Forest fue relativamente pequeña. Esto sugiere que la estrategia de boosting logra aprovechar ligeramente mejor la información disponible, convirtiendo a XGBoost en el modelo con mayor capacidad predictiva para estimar el IMDB Score en este conjunto de datos.